

Examples of Different Kinds of Data to Cache and How to Cache

1. What Can Be Cached?

We can cache two major things:

None

HTTP Responses

Database Query Results

This helps reduce database load, lower latency, and improve scalability.

2. Caching HTTP Responses

We can store full API responses.

Example:

None

GET /products/44

Store response using key:

None

GET:/products/44

Next request returns directly from cache.

Useful for:

- Product pages
- Blog pages
- Public APIs
- Search results

3. Caching Database Queries

Inside backend systems, we cache expensive SQL/NoSQL queries.

Example:

None

```
SELECT * FROM products WHERE id=44
```

Store result in Redis.

Next same query avoids DB hit.

Usually implemented using:

None

```
Cache-Aside Pattern
```

4. Private Cache vs Public Cache

A. Private Cache

Stored on user device/browser.

Used for:

None

User profile page

Personal dashboard

App settings

Recently viewed items

B. Public Cache

Shared across many users.

Stored on:

None

CDN

Reverse Proxy

Redis Layer

API Gateway Cache

Used for common public content.

5. Data That Should NOT Be Cached

Avoid caching:

None

Bank account details

OTP responses

Payment confirmation pages

Private messages

Sensitive tokens

Reason:

None

Security risk

Privacy leakage

6. Realtime Data Usually Poor for Cache

Examples:

None

Stock prices

Live sports score

Flight arrivals now

Hotel room availability today

Because values change rapidly.

Use:

None

Very short TTL

Realtime fetch

Streaming updates

7. Data That Can Be Cached with Validation

Some public data changes sometimes.

Examples:

None

Hotel rooms next month

Flight seats next week

Movie ticket slots

Inventory pages

Use browser/CDN caching with revalidation.

Flow:

None

Client uses cached copy

Checks server freshness

Server returns 304 Not Modified

This saves bandwidth.

8. Important Cache Headers

Use:

None

`max-age=600`

`must-revalidate`

`ETag`

`Last-Modified`

Meaning:

Header	Purpose
max-age	Cache valid for given seconds
must-revalidate	Check server before reuse
ETag	Version identifier
Last-Modified	Timestamp validation

9. Long-Term Cache Data

Some data changes rarely.

Examples:

None

Train schedules

Bus routes

Country list

Static JS/CSS files

Company logo

Images

Use long expiry time.

10. Multi-Layer Caching Strategy

Best systems cache at multiple layers:

None

Browser Cache



CDN Cache



API Gateway Cache



Redis Cache



Database

Goal:

None

Serve request from highest possible layer

11. Example Website Request

User opens product page.

None

1. Browser checks local cache
2. If miss → CDN
3. If miss → API cache
4. If miss → Redis
5. If miss → Database

Very efficient.

12. Why Client Cache is Powerful

Client-side cache saves:

None

- Network latency
- Server CPU
- Database reads
- Bandwidth cost

Especially useful for mobile apps.

13. But Client Cache Has Tradeoff

If client uses cached data:

None

Backend may not know page was opened

So analytics may miss events.

Solution:

None

Client logs usage separately

Send telemetry later

14. Frontend Assets Must Be Cached

Webpage performance improves heavily if cached:

None

CSS

JavaScript

Fonts

Images

Because browser avoids redownload.

15. Example E-Commerce Caching

None

Homepage banners → CDN

Product details → Redis

User cart → Private session cache

Payment APIs → No cache

Inventory count → Short TTL cache

16. Best HLD Interview Answer

If asked caching strategy:

None

Static content → CDN

Frequent DB reads → Redis

Client assets → Browser cache

Sensitive data → no cache

Realtime data → short TTL

Strong practical answer.

17. Advantages

None

Lower latency

Lower DB load

Lower infra cost

Higher throughput

Better UX

18. Disadvantages

None

Stale data

Invalidation complexity

Security mistakes

Missed analytics

Extra infrastructure

19. One-Line Summary

Different data types need different caching strategies: static data gets long cache, changing data needs TTL/revalidation, realtime or sensitive data should avoid caching, and best systems use caching at multiple layers.